



Assignment 2 (Graded)

Technical Assignment: Backend Server Development with Node.js, Express, and MongoDB

Overview

In this assignment, you will build a robust backend using **Node.js**, **Express**, and the **mongodb** package. You will design a schema-like structure for user profiles using 10 distinct data types, implement endpoints to receive and retrieve data, and build filtering capabilities for data retrieval.

Phase 1: Project Plan & Architecture

Before writing code, you must document your architecture. Your project plan should include:

- **API Directory Structure:** A clean layout (e.g., separating server configuration and database connection).
 - **Endpoint Registry:** A list of your 4 endpoints with their corresponding HTTP methods, URI paths, and expected request/response structures.
 - **Database Schema Mapping:** A breakdown of how your user profile object maps to 10 distinct JSON/JavaScript data types.
-

Phase 2: Technical Requirements

1. Database Configuration

- You must use MongoDB package for Mongo Atlas connection and interactions.
- Implement a connection module that establishes a connection to your MongoDB instance online and reuses the database client efficiently across your application.

2. Data Structure Specification (User Profiles), Similar for pet information

Each user document stored in your MongoDB collection must utilize at least **10 different data types**. Use the following blueprint for your document structure:

Field Name Example Value

<code>_id</code>	<code>ObjectId("60c72b2f9b1d8b2bad754f21")</code>
<code>fullName</code>	<code>"Jane Doe"</code>
<code>age</code>	<code>28</code>
<code>hourlyRate</code>	<code>45.50</code>
<code>isActive</code>	<code>true</code>
<code>joinDate</code>	<code>2026-05-15T12:00:00Z</code>
<code>skills</code>	<code>["JavaScript", "Node.js", "Express"]</code>
<code>address</code>	<code>{ "city": "Toronto", "country": "Canada" }</code>

3. API Endpoint Requirements

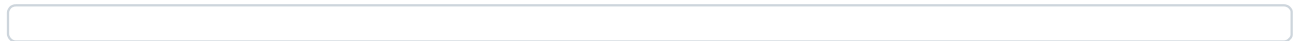
POST Requests (Data Ingestion) example -

- **POST /api/users:** Accepts a single user profile object matching the data specification, processes the inputs, and inserts it into the MongoDB collection.
- **POST /api/pets:** Accepts a single pet related object matching the data specification, processes the inputs, and inserts it into the MongoDB collection.

GET Requests (Data Retrieval & Filtering) example -

- **GET /api/users:** Returns a list of all users from the database.
- **GET /api/users/search:** Implements **query parameter filtering**. The endpoint must parse query strings to filter the MongoDB collection dynamically.
 - *Example:* GET /api/users/search?city=Toronto&age=25 should return only active users living in Toronto who are 25 years old.
 - *Implementation Tip:* Build the MongoDB query object dynamically based on req.query.
- **GET /api/pets:** Returns a list of all pets from the database.

Due date - Submit before the class on Tuesday



0 % 0 of 1 topics complete

Assignment 2 (Graded)



Assignment